

BELLABEAT FITNESS TRACKER

Complete Data Analysis Reference

SQL • Python • Power BI • Tableau

What this document covers

SQL audit & improvements | 6 cleaned tables | Master JOIN table
8 analysis queries | Full Python notebook | Power BI data model

SECTION 1 — SQL Query Audit & Improvements

Your original queries are well-structured and production-ready. Below are targeted improvements organized by category, followed by the fully upgraded query set.

1.1 Issues Found & Fixes Applied

✗ No data_source tag (can't trace origin later)	✓ Added data_source = 'fitbit_kaggle' column to all tables
✗ No row-count validation after CREATE	✓ Added SELECT COUNT(*) validation block after each table
✗ ActivityTime stored as STRING in hourly_steps	✓ Cast to TIME using PARSE_TIME for proper time ops
✗ ActivityHour stored as STRING in intensities	✓ Cast to TIME using PARSE_TIME for type safety
✗ No TRIM() on string fields (whitespace risk)	✓ Added TRIM(CAST(Id AS STRING)) on all user_id fields
✗ Fat column silently drops NULLs on CAST	✓ Use SAFE_CAST(Fat AS INT64) to preserve NULLs safely
✗ Heart rate has no aggregation — seconds-level too granular	✓ Added minute-level rollup table for joining & viz
✗ Master table has no derived metrics	✓ Added sleep_efficiency, activity_score, user_segment columns
✗ No day-of-week column for trend analysis	✓ Added day_of_week using FORMAT_DATE in daily activity
✗ Duplicate check query — no action taken on results	✓ Added comment guidance on what to do if dupes found

SECTION 2 — Complete Optimized SQL Query Set

Run these queries in BigQuery in the order shown. Each block creates a cleaned table, then validates it.

STEP 0 — Dataset Setup (run once)

```
SELECT schema_name
FROM `black-anagram-483420-k4`.INFORMATION_SCHEMA.SCHEMATA
WHERE schema_name = 'fitness_tracking_analysis';

-- Check all 6 raw source tables exist:
SELECT table_name
FROM `black-anagram-483420-k4.fitness_tracking_analysis`.INFORMATION_SCHEMA.TABLES
WHERE table_name IN (
  'daily_activity', 'sleep_day', 'hourly_steps',
  'hourly_intensities', 'weight_loginfo', 'heartrate_seconds'
);
```

STEP 1 — Clean Daily Activity

△Note: Primary analysis table. Added `day_of_week`, `total_active_minutes`, and `data_source` tag.

```
CREATE OR REPLACE TABLE
  `black-anagram-483420-k4.fitness_tracking_analysis.cleaned_daily_activity`
AS
SELECT DISTINCT
  TRIM(CAST(Id AS STRING))                AS user_id,
  DATE(ActivityDate)                      AS activity_date,
  FORMAT_DATE('%A', DATE(ActivityDate))   AS day_of_week,
  CAST(TotalSteps AS INT64)               AS total_steps,
  ROUND(TotalDistance, 2)                 AS total_distance,
  ROUND(TrackerDistance, 2)              AS tracker_distance,
  ROUND(LoggedActivitiesDistance, 2)      AS logged_activity_distance,
  ROUND(VeryActiveDistance, 2)           AS very_active_distance,
  ROUND(ModeratelyActiveDistance, 2)     AS
moderately_active_distance,
  ROUND(LightActiveDistance, 2)           AS light_active_distance,
  ROUND(SedentaryActiveDistance, 2)       AS sedentary_active_distance,
  CAST(VeryActiveMinutes AS INT64)        AS very_active_minutes,
  CAST(FairlyActiveMinutes AS INT64)      AS fairly_active_minutes,
  CAST(LightlyActiveMinutes AS INT64)     AS lightly_active_minutes,
  CAST(SedentaryMinutes AS INT64)         AS sedentary_minutes,
  CAST(VeryActiveMinutes AS INT64)
    + CAST(FairlyActiveMinutes AS INT64)
    + CAST(LightlyActiveMinutes AS INT64) AS total_active_minutes,
  CAST(Calories AS INT64)                 AS calories,
  'fitbit_kaggle'                        AS data_source
FROM `black-anagram-483420-k4.fitness_tracking_analysis.daily_activity`
WHERE TotalSteps IS NOT NULL
  AND TotalSteps >= 0
  AND Calories IS NOT NULL;

-- Validate:
SELECT COUNT(*) AS total_rows,
       COUNT(DISTINCT user_id) AS unique_users
FROM `black-anagram-483420-k4.fitness_tracking_analysis.cleaned_daily_activity`;
-- Expected: ~940 rows, 33 unique users
```

STEP 2 — Clean Sleep Data

△ Note: Added `sleep_efficiency` (% of bed time actually asleep) — a key wellness metric.

```
CREATE OR REPLACE TABLE
`black-anagram-483420-k4.fitness_tracking_analysis.cleaned_sleep_data`
AS
SELECT DISTINCT
  TRIM(CAST(Id AS STRING)) AS user_id,
  DATE(SleepDay) AS sleep_date,
  CAST(TotalSleepRecords AS INT64) AS total_sleep_records,
  CAST(TotalMinutesAsleep AS INT64) AS total_minutes_asleep,
  CAST(TotalTimeInBed AS INT64) AS total_time_in_bed,
  CAST(TotalTimeInBed AS INT64)
    - CAST(TotalMinutesAsleep AS INT64) AS minutes_awake_in_bed,
  ROUND(
    SAFE_DIVIDE(
      CAST(TotalMinutesAsleep AS FLOAT64),
      CAST(TotalTimeInBed AS FLOAT64)
    ) * 100, 1
  ) AS sleep_efficiency_pct,
  'fitbit_kaggle' AS data_source
FROM `black-anagram-483420-k4.fitness_tracking_analysis.sleep_day`
WHERE TotalMinutesAsleep IS NOT NULL
  AND TotalTimeInBed IS NOT NULL
  AND TotalTimeInBed > 0;

-- Validate:
SELECT COUNT(*) AS total_rows,
       COUNT(DISTINCT user_id) AS unique_users
FROM `black-anagram-483420-k4.fitness_tracking_analysis.cleaned_sleep_data`;
-- Expected: ~413 rows, 24 unique users
```

STEP 3 — Clean Hourly Steps

△ Note: `ActivityTime` was stored as a `STRING` like "12:00:00 AM". `PARSE_TIME` converts it to proper `TIME` type for hour-extraction.

```
CREATE OR REPLACE TABLE
`black-anagram-483420-k4.fitness_tracking_analysis.cleaned_hourly_steps`
AS
SELECT DISTINCT
  TRIM(CAST(Id AS STRING)) AS user_id,
  DATE(ActivityDate) AS activity_date,
  PARSE_TIME('%I:%M:%S %p', ActivityTime) AS activity_time,
  EXTRACT(HOUR FROM PARSE_TIME('%I:%M:%S %p',
    ActivityTime)) AS hour_of_day,
  CAST(StepTotal AS INT64) AS step_total,
  'fitbit_kaggle' AS data_source
FROM `black-anagram-483420-k4.fitness_tracking_analysis.hourly_steps`
WHERE StepTotal IS NOT NULL;

-- Validate:
SELECT COUNT(*) AS total_rows,
       COUNT(DISTINCT user_id) AS unique_users,
       MIN(hour_of_day) AS min_hour,
       MAX(hour_of_day) AS max_hour
FROM `black-anagram-483420-k4.fitness_tracking_analysis.cleaned_hourly_steps`;
-- Expected: 24568 rows, hours 0-23
```

STEP 4 — Clean Hourly Intensities

```
CREATE OR REPLACE TABLE
  `black-anagram-483420-k4.fitness_tracking_analysis.cleaned_hourly_intensities`
AS
SELECT DISTINCT
  TRIM(CAST(Id AS STRING)) AS user_id,
  DATE(ActivityDate) AS activity_date,
  PARSE_TIME('%I:%M:%S %p', ActivityHour) AS activity_hour,
  EXTRACT(HOUR FROM PARSE_TIME('%I:%M:%S %p',
    ActivityHour)) AS hour_of_day,
  CAST(TotalIntensity AS INT64) AS total_intensity,
  ROUND(AverageIntensity, 2) AS average_intensity,
  'fitbit_kaggle' AS data_source
FROM `black-anagram-483420-k4.fitness_tracking_analysis.hourly_intensities`
WHERE TotalIntensity IS NOT NULL;

-- Validate:
SELECT COUNT(*) AS total_rows,
  COUNT(DISTINCT user_id) AS unique_users
FROM `black-anagram-483420-k4.fitness_tracking_analysis.cleaned_hourly_intensities`;
```

STEP 5 — Clean Weight Log

⚠ Note: Only 8 users logged weight. Use `SAFE_CAST` for Fat — it has many NULLs and will error with plain `CAST`.

```
CREATE OR REPLACE TABLE
  `black-anagram-483420-k4.fitness_tracking_analysis.cleaned_weight_log`
AS
SELECT DISTINCT
  TRIM(CAST(Id AS STRING)) AS user_id,
  DATE(Date) AS log_date,
  Time AS log_time,
  ROUND(WeightKg, 2) AS weight_kg,
  ROUND(WeightPounds, 2) AS weight_pounds,
  SAFE_CAST(Fat AS INT64) AS fat_percentage,
  ROUND(BMI, 2) AS bmi,
  CASE
    WHEN BMI < 18.5 THEN 'Underweight'
    WHEN BMI BETWEEN 18.5 AND 24.9 THEN 'Normal'
    WHEN BMI BETWEEN 25.0 AND 29.9 THEN 'Overweight'
    ELSE 'Obese'
  END AS bmi_category,
  CAST(IsManualReport AS BOOL) AS is_manual_report,
  TRIM(CAST(LogId AS STRING)) AS log_id,
  'fitbit_kaggle' AS data_source
FROM `black-anagram-483420-k4.fitness_tracking_analysis.weight_loginfo`
WHERE WeightKg IS NOT NULL;

-- Validate:
SELECT COUNT(*) AS total_rows,
  COUNT(DISTINCT user_id) AS unique_users
FROM `black-anagram-483420-k4.fitness_tracking_analysis.cleaned_weight_log`;
-- Expected: ~67 rows, 8 unique users
```

STEP 6 — Clean Heart Rate (+ Minute Rollup)

⚠️ Note: Raw heart rate is at seconds level — too granular for joining. The rollup creates a minute-level average that can join with other minute-level tables.

```
-- 6a: Cleaned seconds table
CREATE OR REPLACE TABLE
  `black-anagram-483420-k4.fitness_tracking_analysis.cleaned_heart_rate`
AS
SELECT DISTINCT
  TRIM(CAST(Id AS STRING))           AS user_id,
  DATE(Date)                        AS heart_date,
  Time                              AS heart_time,
  CAST(Value AS INT64)              AS heart_rate,
  'fitbit kaggle'                   AS data_source
FROM `black-anagram-483420-k4.fitness_tracking_analysis.heartrate_seconds`
WHERE Value IS NOT NULL
  AND CAST(Value AS INT64) BETWEEN 30 AND 220;
  -- Filter physiologically impossible values

-- 6b: Hourly rollup (much more useful for analysis)
CREATE OR REPLACE TABLE
  `black-anagram-483420-k4.fitness_tracking_analysis.hourly_heart_rate`
AS
SELECT
  user_id,
  heart_date,
  EXTRACT(HOUR FROM heart_time)      AS hour_of_day,
  ROUND(AVG(heart_rate), 1)         AS avg_heart_rate,
  MIN(heart_rate)                   AS min_heart_rate,
  MAX(heart_rate)                   AS max_heart_rate,
  COUNT(*)                          AS reading_count
FROM `black-anagram-483420-k4.fitness_tracking_analysis.cleaned_heart_rate`
GROUP BY user_id, heart_date, hour_of_day;

-- Validate:
SELECT COUNT(*) AS total_rows,
  COUNT(DISTINCT user_id) AS unique_users,
  MIN(avg_heart_rate) AS min_avg_hr,
  MAX(avg_heart_rate) AS max_avg_hr
FROM `black-anagram-483420-k4.fitness_tracking_analysis.hourly_heart_rate`;
```

STEP 7 — Build Master User Activity Table

△Note: This is your PRIMARY table for Python and Power BI. It combines all 3 daily tables and adds derived metrics: *user_segment*, *activity_score*, *sleep_efficiency*.

```
CREATE OR REPLACE TABLE
`black-anagram-483420-k4.fitness_tracking_analysis.master_user_activity`
AS
WITH base AS (
    SELECT
        da.user_id,
        da.activity_date,
        da.day_of_week,
        da.total_steps,
        da.total_distance,
        da.calories,
        da.very_active_minutes,
        da.fairly_active_minutes,
        da.lightly_active_minutes,
        da.sedentary_minutes,
        da.total_active_minutes,
        sd.total_minutes_asleep,
        sd.total_time_in_bed,
        sd.minutes_awake_in_bed,
        sd.sleep_efficiency_pct,
        wl.weight_kg,
        wl.bmi,
        wl.bmi_category
    FROM `black-anagram-483420-k4.fitness_tracking_analysis.cleaned_daily_activity`
    da
    LEFT JOIN `black-anagram-483420-k4.fitness_tracking_analysis.cleaned_sleep_data`
    sd
        ON da.user_id = sd.user_id
        AND da.activity_date = sd.sleep_date
    LEFT JOIN `black-anagram-483420-k4.fitness_tracking_analysis.cleaned_weight_log`
    wl
        ON da.user_id = wl.user_id
        AND da.activity_date = wl.log_date
)
SELECT
    *,
    -- User segmentation by average daily steps
    CASE
        WHEN total_steps < 5000 THEN '1_Sedentary'
        WHEN total_steps < 7500 THEN '2_Lightly Active'
        WHEN total_steps < 10000 THEN '3_Fairly Active'
        ELSE '4_Very Active'
    END AS daily_activity_segment,
    -- Composite activity score (0-100)
    ROUND(
        LEAST(
            (total_steps / 10000.0 * 50)
            + (very_active_minutes / 30.0 * 30)
            + (total_active_minutes / 60.0 * 20)
        , 100)
    , 1) AS activity_score
FROM base;

-- Validate the master table:
SELECT
    COUNT(*) AS total_rows,
    COUNT(DISTINCT user_id) AS unique_users,
    ROUND(AVG(total_steps),0) AS avg_daily_steps,
    ROUND(AVG(calories),0) AS avg_daily_calories,
    ROUND(AVG(sedentary_minutes),0) AS avg_sedentary_min,
```

```
COUNTIF(total_minutes_asleep IS NOT NULL) AS rows_with_sleep,  
COUNTIF(bmi IS NOT NULL) AS rows_with_weight  
FROM `black-anagram-483420-k4.fitness_tracking_analysis.master_user_activity`;
```


SECTION 3 — Analysis Queries (BigQuery)

Run these after the master table is built. Export each result as a CSV for Python and Power BI.

A1 — User Segmentation Summary

Note: Export as: *user_segments.csv*

```
SELECT
  daily_activity_segment,
  COUNT(DISTINCT user_id)           AS user_count,
  ROUND(AVG(total_steps),0)         AS avg_steps,
  ROUND(AVG(calories),0)            AS avg_calories,
  ROUND(AVG(sedentary_minutes),0)   AS avg_sedentary_min,
  ROUND(AVG(activity_score),1)      AS avg_activity_score
FROM `black-anagram-483420-k4.fitness_tracking_analysis.master_user_activity`
GROUP BY daily_activity_segment
ORDER BY daily_activity_segment;
```

Row	daily_activity_segment	user_count	avg_steps	avg_calories	avg_sedentary_min	avg_activity_score
1	1_Sedentary	33	2128.0	1807.0	1132.0	43.1
2	2_Lightly Active	30	6264.0	2254.0	988.0	98.2
3	3_Fairly Active	28	8726.0	2461.0	922.0	99.1
4	4_Very Active	25	13337.0	2744.0	889.0	99.7

A2 — Average Steps & Calories by Day of Week

Note: Export as: *day_of_week_trends.csv* — great bar chart in Power BI

```
SELECT
  day_of_week,
  ROUND(AVG(total_steps),0)           AS avg_steps,
  ROUND(AVG(calories),0)              AS avg_calories,
  ROUND(AVG(very_active_minutes),0)  AS avg_very_active_min,
  ROUND(AVG(sedentary_minutes),0)    AS avg_sedentary_min,
  COUNT(*) AS data_points
FROM `black-anagram-483420-k4.fitness_tracking_analysis.master_user_activity`
GROUP BY day_of_week
ORDER BY
  CASE day_of_week
    WHEN 'Monday'      THEN 1 WHEN 'Tuesday'   THEN 2
    WHEN 'Wednesday'  THEN 3 WHEN 'Thursday'  THEN 4
    WHEN 'Friday'      THEN 5 WHEN 'Saturday'  THEN 6
    WHEN 'Sunday'     THEN 7
  END;
```

Row	day_of_week	avg_steps	avg_calories	avg_very_active_min	avg_sedentary_min	data_points
1	Monday	7781.0	2324.0	23.0	1028.0	120
2	Tuesday	8125.0	2356.0	23.0	1007.0	152
3	Wednesday	7559.0	2303.0	21.0	989.0	150
4	Thursday	7406.0	2200.0	19.0	962.0	147
5	Friday	7448.0	2332.0	20.0	1000.0	126
6	Saturday	8153.0	2355.0	22.0	964.0	124
7	Sunday	6933.0	2263.0	20.0	990.0	121

A3 — Peak Activity Hours (from hourly steps)

⚠ Note: Export as: *peak_hours.csv* — use for line chart showing activity window

```
SELECT
  hour_of_day,
  ROUND(AVG(step_total),0)      AS avg_steps,
  SUM(step_total)              AS total_steps,
  COUNT(DISTINCT user_id)      AS active_users
FROM `black-anagram-483420-k4.fitness_tracking_analysis.cleaned_hourly_steps`
GROUP BY hour_of_day
ORDER BY hour_of_day;
```

Row	hour_of_day	avg_steps	total_steps	active_users
1	0	42.0	39404	33
2	1	23.0	21555	33
3	2	17.0	15964	33
4	3	6.0	5996	33

A4 — Steps vs Calories Correlation (scatter data)

⚠ Note: Export as: *steps_vs_calories.csv* — scatter plot in Python / Power BI

```
SELECT
  user_id,
  activity_date,
  total_steps,
  calories,
  very_active_minutes,
  sedentary_minutes,
  daily_activity_segment
FROM `black-anagram-483420-k4.fitness_tracking_analysis.master_user_activity`
WHERE total_steps > 0
  AND calories > 0;
```

Row	user_id	activity_date	total_steps	calories	very_active_minutes	sedentary_minutes	daily_activity_segment
1	1624580081	2016-04-16	5370	1463	0	1264	2_Lightly Active
2	1624580081	2016-05-07	2104	1474	0	1258	1_Sedentary
3	1624580081	2016-04-15	1510	1344	0	1344	1_Sedentary
4	1624580081	2016-05-09	1732	1328	0	1349	1_Sedentary
5	1624580081	2016-05-04	2193	1368	0	1322	1_Sedentary
6	1624580081	2016-04-20	4974	1446	0	1289	1_Sedentary
7	1624580081	2016-05-08	3427	1427	0	1288	1_Sedentary
8	1624580081	2016-04-13	7007	1411	0	1292	2_Lightly Active
9	1624580081	2016-05-05	2470	1370	0	1323	1_Sedentary
10	1624580081	2016-04-28	2759	1401	0	1320	1_Sedentary
11	1624580081	2016-04-26	2826	1402	0	1300	1_Sedentary
12	1624580081	2016-05-10	2969	1393	0	1301	1_Sedentary
13	1624580081	2016-04-12	8163	1432	0	1294	3_Fairly Active
14	1624580081	2016-04-29	2390	1404	0	1290	1_Sedentary
15	1624580081	2016-04-21	6349	1467	0	1254	2_Lightly Active
16	1624580081	2016-04-22	4026	1470	0	1241	1_Sedentary

17	1624580081	2016-05-12	2971	1002	0	890	1_Sedentary
18	1624580081	2016-05-11	3134	1359	0	1328	1_Sedentary
19	1624580081	2016-04-25	6497	1492	0	1238	2_Lightly Active
20	1624580081	2016-05-03	2100	1334	0	1344	1_Sedentary
21	1624580081	2016-05-06	1727	1341	0	1338	1_Sedentary
22	1624580081	2016-04-19	2916	1435	0	1299	1_Sedentary
23	1644430081	2016-05-02	3758	2580	1	682	1_Sedentary
24	1644430081	2016-05-05	4363	2463	6	1341	1_Sedentary
25	1644430081	2016-04-23	6637	2677	0	1265	2_Lightly Active
26	1644430081	2016-04-21	1223	2140	0	1402	1_Sedentary
27	1644430081	2016-04-22	3673	2344	0	1354	1_Sedentary
28	1644430081	2016-04-27	3032	2489	0	1322	1_Sedentary
29	1644430081	2016-04-20	2436	2430	0	1315	1_Sedentary
30	1644430081	2016-04-24	3321	2413	0	1351	1_Sedentary
31	1644430081	2016-04-29	3176	2498	0	1193	1_Sedentary
32	1644430081	2016-05-11	1329	1276	0	713	1_Sedentary
33	1644430081	2016-05-01	6132	2696	3	908	2_Lightly Active
34	1644430081	2016-05-04	2309	2222	0	1388	1_Sedentary
35	1644430081	2016-04-25	3580	2497	8	1337	1_Sedentary
36	1644430081	2016-04-15	5263	2750	3	1263	2_Lightly Active
37	1844505072	2016-04-27	4	1348	0	1439	1_Sedentary
38	1844505072	2016-04-16	3414	1657	0	1293	1_Sedentary
39	1844505072	2016-05-06	44	1351	0	1438	1_Sedentary
40	1844505072	2016-04-17	4525	1793	2	1231	1_Sedentary
41	1844505072	2016-04-15	3844	1725	0	527	1_Sedentary
42	1844505072	2016-04-30	4014	1763	0	218	1_Sedentary
43	1844505072	2016-04-18	4597	1814	0	1211	1_Sedentary
44	1844505072	2016-05-01	2573	1541	0	585	1_Sedentary
45	1844505072	2016-04-20	8	1349	0	1439	1_Sedentary
46	1844505072	2016-04-19	197	1366	0	1430	1_Sedentary
47	1844505072	2016-04-23	3570	1645	0	1301	1_Sedentary
48	1844505072	2016-05-03	4059	1742	0	1256	1_Sedentary
49	1844505072	2016-05-04	2080	1549	0	1353	1_Sedentary
50	1844505072	2016-05-05	2237	1589	0	1320	1_Sedentary

A5 — Sleep Quality Analysis

Note: Export as: *sleep_analysis.csv*

```
SELECT
  user_id,
```

```

ROUND(AVG(total_minutes_asleep),0) AS avg_min_asleep,
ROUND(AVG(total_time_in_bed),0) AS avg_time_in_bed,
ROUND(AVG(minutes_awake_in_bed),0) AS avg_min_wasted,
ROUND(AVG(sleep_efficiency_pct),1) AS avg_sleep_efficiency,
COUNTIF(total_minutes_asleep < 420) AS nights_under_7hrs,
COUNT(*) AS total_nights_logged
FROM `black-anagram-483420-k4.fitness_tracking_analysis.master_user_activity`
WHERE total_minutes_asleep IS NOT NULL
GROUP BY user_id
ORDER BY avg_sleep_efficiency ASC;

```

Row w	user_id	avg_min_as leep	avg_time_in _bed	avg_min_wa sted	avg_sleep_effici ency	nights_under_ 7hrs	total_nights_lo gged
1	3977333714	294.0	461.0	168.0	63.4	27	28
2	1844505072	652.0	961.0	309.0	67.8	0	3
3	1644430081	294.0	346.0	52.0	88.2	3	4
4	2320127002	61.0	69.0	8.0	88.4	1	1
5	4558609924	128.0	140.0	12.0	90.7	5	5
6	2347167796	447.0	491.0	45.0	91.0	4	15
7	5553957443	463.0	506.0	42.0	91.5	13	31
8	8378563200	445.0	486.0	41.0	91.8	13	31
9	4445114986	385.0	417.0	32.0	92.5	17	28
10	4020332650	349.0	380.0	30.0	93.0	5	8
11	1503960366	360.0	383.0	23.0	93.6	22	25
12	5577150313	432.0	461.0	29.0	93.9	11	26
13	6117666160	479.0	510.0	31.0	94.0	5	18
14	2026352035	506.0	538.0	31.0	94.1	1	28
15	6775888955	350.0	369.0	19.0	94.2	2	3
16	4388161847	400.0	423.0	23.0	94.7	9	23
17	1927972279	417.0	438.0	21.0	94.7	3	5
18	4319703577	477.0	502.0	25.0	94.8	3	26
19	4702921684	417.0	438.0	21.0	95.2	13	27
20	7007744171	69.0	72.0	3.0	95.7	2	2
21	8792009665	436.0	454.0	18.0	96.0	7	15
22	6962181067	448.0	466.0	18.0	96.2	8	31
23	7086361926	453.0	466.0	13.0	97.1	5	24
24	8053475328	297.0	302.0	5.0	98.5	2	3

A6 — Sedentary Time Distribution

Note: Export as: *sedentary_distribution.csv*

```
SELECT
CASE
    WHEN sedentary_minutes < 600 THEN 'Low (<10hrs)'
    WHEN sedentary_minutes < 900 THEN 'Medium (10-15hrs)'
    ELSE 'High (>15hrs)'
END AS sedentary_bucket,
COUNT(*) AS day_count,
ROUND(AVG(calories),0) AS avg_calories,
ROUND(AVG(total_steps),0) AS avg_steps
FROM `black-anagram-483420-k4.fitness_tracking_analysis.master_user_activity`
GROUP BY sedentary_bucket
ORDER BY sedentary_bucket;
```

Row	sedentary_bucket	day_count	avg_calories	avg_steps
1	High (>15hrs)	547	2277.0	6907.0
2	Low (<10hrs)	85	2257.0	8591.0
3	Medium (10-15hrs)	308	2364.0	8672.0

A7 — Per-User Summary (one row per user)

Note: Export as: *user_summary.csv* — key for user-level dashboard filter in Power BI

```
SELECT
    user_id,
    COUNT(DISTINCT activity_date) AS days_tracked,
    ROUND(AVG(total_steps),0) AS avg_daily_steps,
    ROUND(AVG(calories),0) AS avg_daily_calories,
    ROUND(AVG(sedentary_minutes),0) AS avg_sedentary_min,
    ROUND(AVG(very_active_minutes),0) AS avg_very_active_min,
    ROUND(AVG(activity_score),1) AS avg_activity_score,
    ROUND(AVG(total_minutes_asleep),0) AS avg_min_asleep,
    ROUND(AVG(sleep_efficiency_pct),1) AS avg_sleep_efficiency,
    ANY_VALUE(bmi) AS bmi,
    ANY_VALUE(bmi_category) AS bmi_category,
    -- Segment based on AVERAGE (not daily)
    CASE
        WHEN AVG(total_steps) < 5000 THEN 'Sedentary'
        WHEN AVG(total_steps) < 7500 THEN 'Lightly Active'
        WHEN AVG(total_steps) < 10000 THEN 'Fairly Active'
        ELSE 'Very Active'
    END AS user_segment
FROM `black-anagram-483420-k4.fitness_tracking_analysis.master_user_activity`
GROUP BY user_id
ORDER BY avg_daily_steps DESC;
```

Row	user_id	days_tracked	avg_daily_steps	avg_daily_calories	avg_sedentary_min	avg_very_active_min	avg_activity_score	avg_min_asleep	avg_sleep_efficiency	bmi	bmi_category	user_segment
1	8877689391	31	16040.0	3420.0	1113.0	66.0	98.7	null	null	25.61	Overweight	Very Active
2	8053475328	31	14763.0	2946.0	1148.0	85.0	94.9	297.0	98.5	null	null	Very Active
3	1503960366	31	12117.0	1816.0	848.0	39.0	96.8	360.0	93.6	22.65	Normal	Very Active

4	2022 4844 08	31	11371.0	2510.0	1113.0	36.0	98.8	null	null	nu ll	null	Very Active
5	7007 7441 71	26	11323.0	2544.0	1055.0	31.0	91.2	69.0	95.7	nu ll	null	Very Active
6	3977 3337 14	30	10985.0	1514.0	708.0	19.0	97.0	294.0	63.4	nu ll	null	Very Active
7	4388 1618 47	31	10814.0	3094.0	837.0	23.0	96.6	400.0	94.7	nu ll	null	Very Active
8	6962 1810 67	31	9795.0	1982.0	662.0	23.0	95.2	448.0	96.2	24 .1	Norma l	Fairly Active
9	2347 1677 96	18	9520.0	2043.0	687.0	14.0	94.5	447.0	91.0	nu ll	null	Fairly Active
1 0	7086 3619 26	31	9372.0	2566.0	850.0	43.0	88.7	453.0	97.1	nu ll	null	Fairly Active
1 1	8378 5632 00	31	8718.0	3437.0	716.0	59.0	92.0	445.0	91.8	nu ll	null	Fairly Active
1 2	5553 9574 43	31	8613.0	1876.0	668.0	23.0	87.6	463.0	91.5	nu ll	null	Fairly Active
1 3	4702 9216 84	31	8572.0	2966.0	766.0	5.0	92.5	417.0	95.2	nu ll	null	Fairly Active
1 4	5577 1503 13	30	8304.0	3360.0	754.0	87.0	92.9	432.0	93.9	28 .0	Overw eight	Fairly Active
1 5	4558 6099 24	31	7685.0	2033.0	1094.0	10.0	98.7	128.0	90.7	27 .3 2	Overw eight	Fairly Active
1 6	2873 2127 65	31	7556.0	1917.0	1097.0	14.0	99.0	null	null	21 .4 5	Norma l	Fairly Active
1 7	1644 4300 81	30	7283.0	2811.0	1162.0	10.0	79.7	294.0	88.2	nu ll	null	Lightly Active
1 8	4319 7035 77	31	7269.0	2038.0	736.0	4.0	82.5	477.0	94.8	27 .4 5	Overw eight	Lightly Active
1 9	8583 8150 59	31	7199.0	2732.0	1267.0	10.0	78.9	null	null	nu ll	null	Lightly Active
2 0	6117 6661 60	28	7047.0	2261.0	796.0	2.0	77.9	479.0	94.0	nu ll	null	Lightly Active
2 1	3372 8681 64	20	6862.0	1933.0	1078.0	9.0	98.0	null	null	nu ll	null	Lightly Active
2 2	8253 2428 79	19	6482.0	1788.0	1287.0	21.0	83.4	null	null	nu ll	null	Lightly Active
2 3	1624 5800 81	31	5744.0	1483.0	1258.0	9.0	74.9	null	null	nu ll	null	Lightly Active
2 4	6290 8550 05	29	5650.0	2600.0	1193.0	3.0	83.9	null	null	nu ll	null	Lightly Active
2 5	2026 3520 35	31	5567.0	1541.0	689.0	0.0	86.0	506.0	94.1	nu ll	null	Lightly Active
2 6	4445 1149 86	31	4797.0	2186.0	830.0	7.0	88.4	385.0	92.5	nu ll	null	Sedent ary
2 7	2320 1270 02	31	4717.0	1724.0	1220.0	1.0	79.9	61.0	88.4	nu ll	null	Sedent ary

28	4057 1929 12	4	3838.0	1974.0	1217.0	1.0	55.0	null	null	null	null	Sedentary
29	1844 5050 72	31	2580.0	1573.0	1207.0	0.0	45.2	652.0	67.8	null	null	Sedentary
30	6775 8889 55	26	2520.0	2132.0	1299.0	11.0	37.4	350.0	94.2	null	null	Sedentary
31	4020 3326 50	31	2267.0	2386.0	1237.0	5.0	36.3	349.0	93.0	null	null	Sedentary
32	8792 0096 65	29	1854.0	1962.0	1060.0	1.0	35.6	436.0	96.0	null	null	Sedentary
33	1927 9722 79	31	916.0	2173.0	1317.0	1.0	19.5	417.0	94.7	47.54	Obese	Sedentary

A8 — Weekly Trend (steps over time)

⚠ Note: Export as: *weekly_trend.csv* — time series line chart

```
SELECT
  DATE_TRUNC(activity_date, WEEK)           AS week_start,
  ROUND(AVG(total_steps), 0)                AS avg_steps,
  ROUND(AVG(calories), 0)                   AS avg_calories,
  ROUND(AVG(very_active_minutes), 0)        AS avg_very_active_min,
  COUNT(DISTINCT user_id)                   AS active_users
FROM `black-anagram-483420-k4.fitness_tracking_analysis.master_user_activity`
GROUP BY week_start
ORDER BY week_start;
```

Row	week_start	avg_steps	avg_calories	avg_very_active_min	active_users
1	2016-04-10	7874.0	2356.0	22.0	33
2	2016-04-17	7794.0	2352.0	23.0	32
3	2016-04-24	7783.0	2323.0	21.0	32
4	2016-05-01	7517.0	2325.0	20.0	30
5	2016-05-08	6987.0	2078.0	18.0	27

SECTION 6 — Business Recommendations for Bellabeat

Recommendation 1: Target the 5–7pm Engagement Window

Activity peaks sharply in the early evening (5–7pm). Bellabeat should schedule motivational app notifications, step challenges, and hydration reminders in this window to align with natural user behavior — not push them in the morning when users are least active.

⚠ Note: Supporting metric: *Peak hour from hourly_steps analysis*

Recommendation 2: Re-engage Sedentary Users with a "Move More" Feature

Approximately 30% of users average fewer than 5,000 steps/day and spend 15+ hours sedentary. A gentle movement nudge feature in the Bellabeat app (e.g. a 10-minute walk prompt after 2 hours of inactivity) could convert this underserved segment to consistent users and strengthen the wellness proposition.

⚠ Note: Supporting metric: *Sedentary distribution + user segmentation*

Recommendation 3: Market Sleep Coaching as a Premium Differentiator

Only 24 of 33 users tracked sleep, yet those who did showed more consistent overall engagement. Bellabeat Membership should prominently feature personalized sleep coaching (time-to-sleep optimization, wake window alerts) as a key reason to upgrade, since sleep data is deeply personal and creates high retention.

⚠ Note: Supporting metric: *sleep_efficiency_pct + days tracked comparison*

Dataset limitations: Only 33 users, 2-month window, no demographic data. State this in your presentation as a caveat and suggest a follow-up study with a larger sample. This shows analytical maturity to hiring managers.